Tuples and Sets in Python 3

Module: Core Python · Collections
Previous module: Lists
Next module: Dictionaries

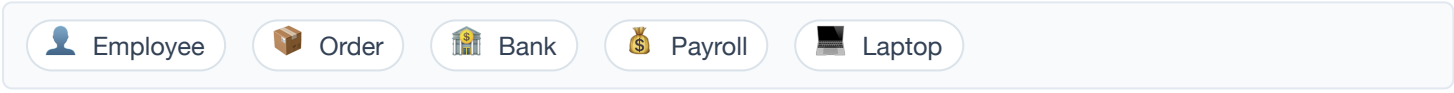
Learning Outcomes

By the end of this session, learners will be able to:

1. Explain **tuple immutability** and when to use tuples vs lists.
2. Create tuples; use **packing** and **unpacking** for clean assignment and function returns.
3. Explain **set uniqueness** and why sets ignore duplicate values.
4. Perform set operations: union, intersection, difference, symmetric difference.
5. Apply tuples and sets to real scenarios: coordinates, employee records, deduplication, tag lists.

Tuples — the sealed envelope

A **tuple** is like a printed receipt or Aadhaar-linked record: once created, you should **not change** the fields. You can **read** them, pass them around, and unpack them into variables.



Where used?
- **AcmeHR** — fixed employee row `(emp_id, name, dept)` snapshots
- **TechMart** — product catalog rows, shipping status tuples
- **SecureBank** — immutable transaction snapshots (date, type, ₹ amount)
- College apps — GPS campus coords, exam slot `(date, room)`
- **MedCare** — appointment keys, unique tag sets for departments

Use tuples when:

- Data should not change accidentally (database row snapshot, GPS point `(lat, lon)`).
- Returning **multiple values** from a function: `return name, salary, dept`.
- Using as **dictionary keys** (lists cannot be keys — not hashable).

Sets — the unique guest list

A **set** is like a club entry register that **automatically rejects duplicates**. If two people with the same employee ID try to enter, only one entry remains.

Use sets when:

- You need **unique** values: unique user IDs, unique tags, unique cities visited.
- You need fast **membership check**: `"Python" in skills_set`.
- You need **math-style** comparisons: students in both batches, products only in warehouse A.

Concepts and Definitions

Term	Definition
Tuple	Ordered, **immutable** sequence written with parentheses `( )`.
Immutability	Cannot change, add, or remove elements after creation.
Packing	Combining values into one tuple: `t = 1, 2, 3`.
Unpacking	Assigning tuple elements to variables: `x, y, z = t`.
Set	Unordered collection of **unique** hashable items; `{}` or `set()`.
Hashable	Object that can be used in sets/dict keys (int, str, tuple of hashables — not list).
Union (`|`)	All elements in either set.
Intersection (`&`)	Elements in both sets.
Difference (`-`)	In first set but not second.
Symmetric difference (`^`)	In one set or the other, not both.
frozenset	Immutable set (advanced; used as dict key).

Step-by-Step Notes with Annotated Python 3 Code

Part A — Tuples

Step 1 — Create and read tuples

```
# CREATE — parentheses optional for packing
employee = ("EMP-101", "Ananya Sharma", "Engineering")
single = (42,)              # comma required for 1-item tuple
not_a_tuple = (42)          # just int 42 — common mistake!

# READ — same indexing/slicing as lists
print(employee[0])          # EMP-101
print(employee[-1])         # Engineering
print(employee[1:3])        # ('Ananya Sharma', 'Engineering')

# IMMUTABLE — this fails:
# employee[0] = "EMP-999"   # TypeError: 'tuple' object does not support item assignment
```

Note:
- Wrong: Writing `(42)` and thinking it is a 1-tuple — it is just an int.
- Prefer: Always use a trailing comma for a 1-tuple: `(42,)`.

Try these (2 programs):

Program 1 — AcmeHR employee tuple by index

```
emp = ("EMP-204", "Priya Nair", "HR")
print(emp[0], emp[1], emp[2])
```

Expected Output:

```
EMP-204 Priya Nair HR
```

Program 2 — One-element tuple vs int

```
one = (100,)
not_tuple = (100)
print(type(one), type(not_tuple))
```

Expected Output:

```
<class 'tuple'> <class 'int'>
```

Common Error: Writing `(100)` thinking it is a 1-tuple — parentheses alone do not make a tuple. Need the comma: `(100,)`.

Step 2 — Tuple methods (only two!)

```
scores = (78, 92, 85, 92, 88)

print(scores.count(92))     # 2
print(scores.index(85))     # 2 — first index of 85
print(len(scores))          # 5
```

Try these (2 programs):

Program 1 — Tuple count and index

```
scores = (78, 92, 85, 92, 88)
print(scores.count(92))
print(scores.index(85))
```

Program 2 — Shipping tuple length

```
shipment = ("ORD-9", 1499.0, "Packed")
print(len(shipment))
```

Step 3 — Packing and unpacking

```
# PACKING — values bundled into tuple
order_summary = "ORD-5001", 2499.00, "Delivered"
print(type(order_summary))  # <class 'tuple'>

# UNPACKING — assign to variables
```

```
order_id, amount, status = order_summary
print(f"Order {order_id}: ₹{amount} — {status}")

# Extended unpacking with *
head, *middle, tail = (1, 2, 3, 4, 5)
print(head, middle, tail)   # 1 [2, 3, 4] 5

# Swap two variables (Pythonic)
a, b = 10, 20
a, b = b, a
print(a, b)                 # 20 10
```

Note:
- Wrong: Unpacking the wrong number of variables → `ValueError: too many/few values`.
- Prefer: Match variable count to tuple length.

Try these (2 programs):

Program 1 — Unpack expense tuple

```
date, category, amount = ("31-07-2026", "Food", 450.0)
print(f"{date} | {category} | ₹{amount}")
```

Expected Output:

```
31-07-2026 | Food | ₹450.0
```

Program 2 — Swap with unpacking

```
a, b = "Ravi", "Priya"
a, b = b, a
print(a, b)
```

Expected Output:

```
Priya Ravi
```

Step 4 — Tuples as function return values

```
def get_employee_info(emp_id):
    # Simulated lookup
    if emp_id == 101:
        return ("Ananya", "Engineering", 85000)   # tuple return
    return None

result = get_employee_info(101)
if result:
    name, dept, salary = result                 # unpack
    print(f"{name} works in {dept}, CTC ₹{salary}")
```

Try these (2 programs):

Program 1 — minmax returns a tuple

```
def minmax(nums):
    return min(nums), max(nums)

lo, hi = minmax([45, 90, 70])
print(lo, hi)
```

Expected Output:

```
45 90
```

Program 2 — split_name returns a tuple

```
def split_name(full):
    first, last = full.split(" ", 1)
    return first, last

first, last = split_name("Ravi Kumar")
print(first, last)
```

Step 5 — Tuple of records (e-commerce shipping)

```
# List of tuples: (product_id, name, price)
catalog = [
    ("P001", "Wireless Mouse", 799),
    ("P002", "Keyboard", 1499),
    ("P003", "USB Hub", 599),
]

for pid, name, price in catalog:                # unpack in loop
    print(f"{pid}: {name} — ₹{price}")
```

Try these (2 programs):

Program 1 — Loop catalog tuples

```
catalog = [
    ("P001", "Wireless Mouse", 799),
    ("P002", "Keyboard", 1499),
    ("P003", "USB Hub", 599),
]
for sku, name, price in catalog:
    print(f"{sku} {name}: ₹{price}")
```

Program 2 — Filter under ₹1000

```
catalog = [
    ("P001", "Wireless Mouse", 799),
    ("P002", "Keyboard", 1499),
    ("P003", "USB Hub", 599),
]
for sku, name, price in catalog:
```

```
    if price < 1000:
        print(f"{sku} {name}: ₹{price}")
```

Part B — Sets

Step 6 — Create sets and uniqueness

```
# CREATE
skills = {"Python", "SQL", "Git", "Python"}     # duplicate removed
empty = set()                                   # NOT {} — that's dict!

# From list — remove duplicates
raw_ids = [101, 102, 101, 103, 102, 104]
unique_ids = set(raw_ids)
print(unique_ids)               # {101, 102, 103, 104} — order not guaranteed

# Membership — fast check
print("Python" in skills)       # True
print("Java" in skills)         # False
```

Note:
- Wrong: Using `{}` for an empty set (that creates an empty **dict**); putting lists inside sets (`TypeError: unhashable`); expecting a fixed set order.
- Prefer: `set()` for empty; use a tuple (or frozenset) instead of a list; use a list when order matters.

Try these (2 programs):

Program 1 — Unique IDs with set

```
ids = [101, 102, 101, 103, 102]
unique = set(ids)
print(len(unique))
```

Expected Output:

```
4
```

Remember: Sets drop duplicates and are **unordered** — never rely on print order for set contents in exams or production logs.

Program 2 — Membership in skill set

```
skills = {"Python", "SQL", "Git"}
print("Python" in skills)
print("Java" in skills)
```

Expected Output:

```
True
False
```

Step 7 — Set operations (teams and products)

```
team_a = {"Ananya", "Dev", "Meera", "Ravi"}
team_b = {"Meera", "Priya", "Ravi", "Kiran"}

# UNION — everyone in either team
all_members = team_a | team_b
# or: team_a.union(team_b)
print(all_members)

# INTERSECTION — in both teams (shared resources)
shared = team_a & team_b
print(shared)                   # {'Meera', 'Ravi'}

# DIFFERENCE — in A but not B
only_a = team_a - team_b
print(only_a)                   # {'Ananya', 'Dev'}

# SYMMETRIC DIFFERENCE — exclusive to one team
exclusive = team_a ^ team_b
print(exclusive)                # {'Ananya', 'Dev', 'Priya', 'Kiran'}
```

Try these (2 programs):

Program 1 — Intersection and difference

```
batch_a = {"Priya", "Ravi", "Dev"}
batch_b = {"Ravi", "Meera"}
print(batch_a & batch_b)
print(batch_a - batch_b)
```

Program 2 — Tag union

```
print({"sale", "new"} | {"gift", "sale"})
```

Step 8 — Modify sets

```
cart_categories = {"Electronics", "Books"}

cart_categories.add("Stationery")             # add one
cart_categories.update(["Home", "Books"])       # add many; Books ignored if present
cart_categories.remove("Books")                 # KeyError if missing
cart_categories.discard("Books")                # no error if missing
removed = cart_categories.pop()                 # removes arbitrary element
print(cart_categories)
```

Note:
- Wrong: `remove` on a missing set item → `KeyError`.
- Prefer: Use `discard` when the item might be absent.

Try these (2 programs):

Program 1 — add and discard

```
cats = {"Electronics"}
cats.add("Books")
```

```
cats.discard("Food")
print(cats)
```

Program 2 — update with a list

```
cats = {"Electronics"}
cats.update(["Home", "Books"])
print(cats)
```

Step 9 — Practical: deduplicate visitor emails

```
signup_emails = [
    "dev@example.com", "aisha@example.com",
    "dev@example.com", "meera@example.com", "aisha@example.com"
]

unique_emails = list(dict.fromkeys(signup_emails))  # preserves order
# Alternative: list(set(signup_emails))               # order not preserved

print(unique_emails)
# ['dev@example.com', 'aisha@example.com', 'meera@example.com']
```

Note:
- Wrong: Treating `set([1, 2, 2])` as a sorted unique list — a set is unordered.
- Prefer: `sorted(set(...))` when you need sorted unique output.

Try these (2 programs):

Program 1 — Deduplicate preserving order

```
emails = ["ravi@ksr.in", "priya@ksr.in", "ravi@ksr.in"]
unique = list(dict.fromkeys(emails))
print(unique)
```

Expected Output:

```
['ravi@ksr.in', 'priya@ksr.in']
```

Program 2 — Deduplicate with set (order may differ)

```
emails = ["ravi@ksr.in", "priya@ksr.in", "ravi@ksr.in"]
print(list(dict.fromkeys(emails)))  # order kept
print(set(emails))  # order not guaranteed
```

Step 10 — Sets with ecommerce tags

```
product_a_tags = {"sale", "electronics", "wireless"}
product_b_tags = {"sale", "gift", "wireless"}

common_tags = product_a_tags & product_b_tags
print(common_tags)              # {'sale', 'wireless'}

# Tags only on product A
```

```
unique_to_a = product_a_tags - product_b_tags
print(unique_to_a)              # {'electronics'}
```

Try these (2 programs):

Program 1 — Common product tags

```
print({"sale", "wireless"} & {"sale", "gift"})
```

Program 2 — Tags only on product A

```
print({"electronics", "sale"} - {"sale", "gift"})
```

Practice Exercises

Basic

1. Create a tuple for one expense: `(date, category, amount)`. Print each field by unpacking.

Model answer:

```
expense = ("2026-08-04", "Food", 450.0)
date, category, amount = expense
print(date)
print(category)
print(amount)
```

2. Create a set from `[1, 2, 2, 3, 3, 3]`. Print length before and after.

Model answer:

```
nums = [1, 2, 2, 3, 3, 3]
print(len(nums))       # 6
unique = set(nums)
print(len(unique))     # 3 → {1, 2, 3}
print(unique)
```

3. Given `t = (10, 20, 30, 40)`, print first and last using unpacking with `*`.

Model answer:

```
t = (10, 20, 30, 40)
first, *middle, last = t
print(first)   # 10
print(last)    # 40
print(middle)  # [20, 30]
```

Intermediate

4. Two lists: `batch_2024 = ["A", "B", "C", "D"]` and `batch_2025 = ["C", "D", "E", "F"]`. Find students in both batches using sets.

Model answer:

```
batch_2024 = ["A", "B", "C", "D"]
batch_2025 = ["C", "D", "E", "F"]
both = set(batch_2024) & set(batch_2025)
print(both)  # {'C', 'D'}
```

5. Write a function `split_name(full_name)` returning `(first, last)` for `"Ananya Sharma"`.

Model answer:

```
def split_name(full_name):
    parts = full_name.strip().split()
    first = parts[0]
    last = parts[-1] if len(parts) > 1 else ""
    return first, last

print(split_name("Ananya Sharma"))  # ('Ananya', 'Sharma')
```

6. Remove duplicates from `product_ids = [101, 102, 101, 103, 102]` preserving first-seen order.

Model answer:

```
product_ids = [101, 102, 101, 103, 102]
seen = set()
ordered = []
for pid in product_ids:
    if pid not in seen:
        seen.add(pid)
        ordered.append(pid)
print(ordered)  # [101, 102, 103]
```

Interview-style

7. Why can a tuple be a dict key but a list cannot? (Hint: hashable / mutable.)

Model answer: Dict keys must be **hashable** (stable hash while in the dict). Tuples of immutables are hashable; lists are **mutable**, so their contents (and hash) could change — Python forbids lists as keys. Example: `{(12.97, 77.59): "Bengaluru office"}` works; `{[12.97, 77.59]: "..."}` raises `TypeError`.

8. Given sets `A = {1,2,3}` and `B = {3,4,5}`, compute `A | B`, `A & B`, `A - B`, `A ^ B` without running code — then verify.

Model answer: Predicted: `|` union `{1,2,3,4,5}`; `&` intersection `{3}`; `-` difference `{1,2}`; `^` symmetric difference `{1,2,4,5}`.

`python A = {1, 2, 3} B = {3, 4, 5} print(A | B) print(A & B) print(A - B) print(A ^ B)`

9. Return min and max from a list using tuple unpacking from a helper function.

Model answer:

```
def min_max(values):
    return min(values), max(values)

lo, hi = min_max([55, 78, 92, 67, 88])
print(lo, hi)  # 55 92
```

Mini-Project (Assignment): Email / Product ID Dedup Tool

Title: Clean duplicate IDs before DB import
Lab kit (build here): 15_Lab_Assets/31_email_id_dedup/
Later (overview): 12_structure_chooser — pick list vs tuple vs set vs dict

What to build

```
def deduplicate(items, preserve_order=True):
    """Remove duplicates from a list of strings or ints."""
    if preserve_order:
        return list(dict.fromkeys(items))
    return list(set(items))

def main():
    print("=== Dedup Tool ===")
    raw = input("Enter IDs comma-separated: ")
    ids = [x.strip() for x in raw.split(",") if x.strip()]
    print("Original count:", len(ids))
    clean = deduplicate(ids)
    print("Unique count:", len(clean))
    print("Clean list:", clean)

if __name__ == "__main__":
    main()
```

Sample:

```
Enter IDs comma-separated: 101, 102, 101, 103, 102, 104
Original count: 6
Unique count: 4
Clean list: ['101', '102', '103', '104']
```

Stretch

- Accept a second mode that does **not** preserve order (`set`).
- Deduplicate emails case-insensitively while keeping the first-seen casing.

Done when

- [ ] Lab kit dedup ticket passes
- [ ] Comma-separated input prints original vs unique counts
- [ ] Order-preserving dedupe matches the sample

Interview Q&A

Shared with students after concepts and the mini-project. Use for review and interview prep — questions and model answers are included.

1. If tuples are immutable, why can a tuple containing a list be "changed"? (Mutable element inside — discuss carefully.)
2. When would you store employee data as tuple vs dict vs list of lists?
3. Why are sets unordered? Does it matter for "unique product tags"?

4. Real apps: where might you use set intersection for recommendations ("customers who bought X also bought Y")?
5. Is `bool` a subclass of `int` affecting `(True, False)` as tuple? (Optional deep dive.)
6. **List vs tuple?**
   A: List mutable, tuple immutable; tuple slightly faster, can be dict key if all elements hashable.
7. **How to merge two sets?**
   A: `a | b` or `a.union(b)`.
8. **Are sets ordered in Python 3.7+?**
   A: No. Sets are **unordered** by language spec — never depend on iteration order. (Dicts keep insertion order since 3.7; that does **not** apply to sets.)
9. **Convert list to tuple and back?**
   A: `tuple(lst)`, `list(t)`.
10. **When use frozenset?**
    A: Immutable set as dict key or set element.
